DESIGN AND ANALYSIS OF ALGORITHMS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE CODE & NAME: CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

ASSESSMENT: CO2 -AT2– DEBUGGING AND OPTIMISATION TASK

TITLE: DEBUGGING AND OPTIMISATION OF SENTINELOPS SOC ALERT-TRIAGE MODULE

Submitted by: S.Harshavardhan

Registration Number:192524106

B.Tech – Artificial Intelligence and Data Science

SIMATS Engineering

1. INTRODUCTION

In today's digital world, cybersecurity plays an important role in protecting computer systems, networks and sensitive information from different types of attacks. Organisations receive a large number of security alerts every day from their servers, applications and network devices. These alerts must be analysed properly to identify suspicious activities and prevent security threats.

SentinelOps is a Security Operations Centre (SOC) module designed to manage security alerts and identify possible cyberattacks. The main purpose of this module is to process incoming security alerts, identify blocked IP addresses, detect attack signatures in log files and find suspicious login activities based on the locations of login attempts.

The given Python program performs four important operations using basic algorithms from Design and Analysis of Algorithms. The first operation is sorting security alerts according to their severity. Alerts with higher severity must appear before alerts with lower severity. If two alerts have the same severity, their original arrival order must be maintained. This property is known as stable sorting.

The second operation is searching for an IP address in a blocklist. A blocklist contains IP addresses that are considered suspicious or harmful. Whenever an incoming request contains an IP address from this list, the system must identify it as blocked. Sequential search is initially used to check each address one by one.

The third operation is finding attack signatures in log files. An attack signature is a specific pattern of characters that may indicate suspicious activity. The brute-force string-matching technique is used to compare the given pattern with different positions in the log file and identify all matching positions.

The fourth operation is finding the closest pair of login locations. This operation helps the security team identify login attempts that occur at nearby locations. The Euclidean distance formula is used to calculate the distance between two login locations. The pair having the minimum distance is selected as the closest pair.

However, the original program contains several logical errors that produce incorrect results. Some security alerts may disappear or get duplicated, blocked IP addresses may not be identified correctly, attack signatures at the end of a log may be missed, and the closest-login function may return an incorrect pair.

Therefore, this assessment focuses on identifying the errors, understanding their root causes, correcting the program step by step, testing the corrected functions and improving the algorithms for better performance.

The final objective is to develop a reliable and optimised Python module that can process approximately 50,000 security alerts and 8,000 login events efficiently while maintaining correct results.

2: OBJECTIVES

The main objective of this assessment is to understand how brute force algorithms work and how debugging techniques can be used to identify and correct errors in a program. The assessment focuses on the SentinelOps Security Operations Centre (SOC) Alert-Triage system, which processes security alerts, checks suspicious IP addresses, searches log files for attack patterns, and identifies the closest login events.

The first objective is to understand the working of different brute force algorithms used in sorting, searching, pattern matching, and closest-pair problems. These algorithms solve problems by checking possible solutions directly. By implementing these techniques, we can understand how an algorithm processes input data and produces the required output.

The second objective is to identify and correct the logical errors present in the given Python programs. Even a small programming mistake can produce incorrect results or cause a program to behave unexpectedly. Therefore, the assessment aims to carefully examine the code, identify the incorrect statements, understand the reasons behind the errors, and modify the code to obtain the expected output.

Another important objective is to understand the importance of test cases in software development. The corrected programs must be tested with different types of input data, including normal cases, empty inputs, repeated values, and boundary conditions. Testing helps confirm that the program works correctly under different situations rather than only for one particular input.

The assessment also aims to improve the performance of algorithms through optimisation techniques. A program may produce the correct result but still take more time when processing large amounts of data. Therefore, suitable optimisation methods are studied to reduce unnecessary operations and improve execution speed. For example, insertion sort is examined for nearly sorted data, sets are used for faster IP address lookups, and efficient pattern-matching techniques are considered for searching log files.

Another objective is to analyse the time complexity and space complexity of the algorithms. Time complexity explains how the execution time increases when the input size becomes larger, while space complexity explains how much additional memory an algorithm requires. Understanding these complexities helps in selecting suitable algorithms for real-world applications.

Finally, this assessment aims to develop practical programming and problem-solving skills. It provides experience in debugging, testing, comparing algorithm performance, writing clean Python code, and documenting the results. These skills are useful for developing reliable and efficient software systems, especially in security applications where incorrect results or slow processing can affect the detection of potential threats.

PAGE 3: PROBLEM STATEMENT

The SentinelOps Security Operations Centre (SOC) Alert-Triage system is designed to monitor security-related activities and identify possible threats in a computer network. Every day, a security system may receive thousands of alerts, IP addresses, and log entries. These records must be processed carefully to identify suspicious activities and help security analysts take appropriate action.

The given SentinelOps system contains four important operations: sorting security alerts based on their severity, searching IP addresses in a blocklist, finding attack signatures in log files, and identifying the closest pair of login events. Each operation uses a basic algorithmic technique to process the given input data.

The first problem is to sort security alerts according to their severity. Each alert contains an alert ID and a severity value. Alerts with higher severity should appear before alerts with lower severity. If two alerts have the same severity, their original order should be maintained. However, the given sorting program contains logical mistakes in its insertion sort implementation, which may cause incorrect ordering and data loss.

The second problem is to check whether an IP address belongs to a suspicious IP address blocklist. The system receives a list of blocked IP addresses and checks whether a particular IP address is present in that list. The program should return True when the address is found and False when it is not found. However, the given implementation contains errors in its return statements, which may produce incorrect results.

The third problem is to search for a particular attack signature in a security log. A log file contains a sequence of characters, and the system must identify every position where the given pattern occurs. The program should also detect a pattern that appears at the very end of the log. However, the given pattern-matching program does not examine all valid starting positions because of an incorrect loop boundary.

The fourth problem is to identify the closest pair of login events. Each login event is represented by two numerical values representing its coordinates. The system must calculate the distance between every possible pair of different login events and identify the pair with the minimum distance. The given program contains multiple logical errors related to distance initialisation, comparison conditions, and pair selection.

These problems create difficulties in obtaining accurate results from the SentinelOps system. Incorrect sorting can affect alert prioritisation, incorrect blocklist searching can fail to identify suspicious IP addresses, incomplete pattern matching can miss possible attack signatures, and incorrect distance calculations can produce unreliable login-event analysis.

Therefore, the main problem addressed in this assessment is to identify all logical errors in the given algorithms, correct the programs, verify their outputs using suitable test cases, and improve their performance wherever possible. The final implementation should produce accurate results and handle different input conditions reliably.

4: A. REPRODUCE THE OUTPUTS AND IDENTIFY THE DEFECTS

A.1 Introduction

The first part of the assessment focuses on executing the given Python programs and checking whether their actual outputs match the expected outputs. The SentinelOps SOC Alert-Triage system contains four different operations: sorting security alerts, searching blocked IP addresses, finding attack signatures in log files, and identifying the closest pair of login events.

Each program is examined separately. The given input values are used to execute the programs, and the actual results are compared with the expected results. If the output is incorrect, the corresponding programming statement is identified as a defect.

The purpose of this process is to understand how logical errors affect program execution and how small mistakes in conditions, loop boundaries, and variable initialisation can produce incorrect results.

A.2 Sorting Security Alerts Using Insertion Sort

Problem Description:

The SentinelOps system receives multiple security alerts with different severity levels. Each alert contains an alert ID and a severity value. The alerts must be arranged in descending order of severity, meaning that the alert with the highest severity should appear first.

Insertion sort is used to arrange the alerts. It works by selecting one element at a time and inserting it into its correct position among the previously processed elements.

Input:

alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]

Expected Output:

[("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]

Explanation:

The alert A2 has the highest severity value of 9, so it should appear first. Alert A4 also has a severity value of 9, so it should appear immediately after A2. The remaining alerts are arranged according to their severity values in descending order.

When two alerts have equal severity, their original order must be maintained. This property is called stable sorting.

Identified Defects:

1. The loop condition uses j > 0 instead of j >= 0. Because of this mistake, the first element is not checked during insertion.
2. The statement alerts[j] = key is incorrect. The correct statement is alerts[j+1] = key, because the selected alert must be inserted into the next correct position.

5: A.3 BLOCKLIST SEARCHING USING BRUTE FORCE TECHNIQUE

Problem Description

The SentinelOps SOC system maintains a blocklist containing suspicious IP addresses. These addresses may be associated with malicious activities, unauthorized access, or security attacks. Whenever a new login request is received, the system checks whether the IP address is present in the blocklist.

The brute force searching technique is used to solve this problem. It checks each IP address in the blocklist one by one until the required address is found or the entire list has been examined.

Input

blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]

Expected Output

10.0.0.9 : True10.0.0.5 : True8.8.8.8 : False

Explanation

The system checks each requested IP address against the blocklist.

For the first query, 10.0.0.9 is present in the blocklist. Therefore, the result should be True.

For the second query, 10.0.0.5 is also present in the blocklist. Therefore, the result should be True.

For the third query, 8.8.8.8 is not present in the blocklist. Therefore, the result should be False.

Identified Defects

Defect 1: Incorrect return statement

The original program contains an incorrect return statement when an IP address matches an element in the blocklist. It returns False instead of True.

Correction:

if address == ip: return True

This statement ensures that the function returns True whenever the required IP address is found.

Defect 2: Incorrect placement of return statement

The original program returns False inside the loop. Because of this, the searching process stops after checking the first IP address if it does not match.

Correction:

def is_blocked(blocklist, ip): for address in blocklist: if address == ip: return True return False

PAGE 6: A.4 ATTACK SIGNATURE SEARCHING IN LOG FILES

Problem Description

The SentinelOps SOC system continuously monitors log files to identify suspicious activities and possible security attacks. An attack signature is a specific sequence of characters that may indicate malicious behaviour.

The system must search for a given pattern in the complete log file and identify all the positions where the pattern occurs.

The brute force pattern-matching technique is used to solve this problem. It compares the pattern with the log file character by character at every possible starting position.

Input

log = "FAILFAIXFAIL"pattern = "FAIL"

Expected Output

[0, 8]

Explanation

The given log contains the following sequence:

FAILFAIXFAIL

The pattern to be searched is FAIL.

The pattern occurs two times in the log:

1. The first occurrence starts at index 0.
2. The second occurrence starts at index 8.

Therefore, the expected output is [0, 8].

Python uses zero-based indexing, so the first character of the log is at position 0.

Identified Defect

Defect 1: Incorrect loop boundary

The original program uses the following loop condition:

for i in range(n - m):

This condition does not check the final valid starting position of the pattern.

Here:

- n represents the length of the log.
- m represents the length of the pattern.

When the pattern occurs at the end of the log, the original loop may fail to identify it.

Correction:

for i in range(n - m + 1):

Adding +1 ensures that the final valid starting position is also checked.

Corrected Code

def find_signature(log, pattern): n = len(log) m = len(pattern) hits = [] if m == 0: return list(range(n + 1)) for i in range(n - m + 1): j = 0 while j < m and log[i + j] == pattern[j]: j += 1 if j == m: hits.append(i) return hitslog = "FAILFAIXFAIL"pattern = "FAIL"print(find_signature(log, pattern))

Result

Output:

[0, 8]

After correcting the loop boundary, the program successfully identifies both occurrences of the attack signature, including the occurrence at the end of the log file.

The brute force pattern-matching technique has a worst-case time complexity of O(n × m), where n is the length of the log and m is the length of the pattern.

7: A.5 CLOSEST PAIR OF LOGIN EVENTS

Problem Description

The SentinelOps SOC system records login events using numerical coordinates. These coordinates represent the location or position associated with each login event.

The main objective is to identify the two login events that have the minimum distance between them. This helps the security system identify login activities that occur very close to each other.

The brute force technique is used to solve this problem. It calculates the distance between every possible pair of login events and selects the pair with the smallest distance.

Input

events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]

Expected Output

Closest Pair: (0, 5)Minimum Distance: 1.4142

Explanation

The given input contains six login events. Each event is represented by an X-coordinate and a Y-coordinate.

The distance between two points is calculated using the Euclidean distance formula.

Distance Formula:

Distance = $\sqrt{((x2 - x1)^2 + (y2 - y1)^2)}$

The program calculates the distance between different pairs of login events.

For example, consider the events at index 0 and index 5.

- Event at index 0 = (2, 3)
- Event at index 5 = (3, 4)

Using the distance formula:

Distance = $\sqrt{((3 - 2)^2 + (4 - 3)^2)}$

Distance = $\sqrt{(1^2 + 1^2)}$

Distance = $\sqrt{2}$

Distance = 1.4142

After comparing the distances between all possible pairs, the minimum distance is found between events at index 0 and index 5.

Identified Defects

The original closest-pair program contains four logical defects.

Defect 1: Incorrect minimum distance initialisation

The program initially assigns the minimum distance as 0. Since distances are positive values, the program may never find a distance smaller than zero.

Correction:

```
min_dist = float('inf')
```

This allows the program to update the minimum distance when it finds a smaller value.

Defect 2: Incorrect comparison operator

The original program uses the greater-than (>) operator instead of the less-than (<) operator.

Correction:

```
if d < min_dist:    min_dist = d
```

This ensures that the program selects the smallest distance.

Defect 3: Comparing an event with itself

The original program includes pairs where both indexes are the same. The distance between an event and itself is always zero, which is not useful for finding the closest pair of different events.

Correction:

The program should compare only different events.

Defect 4: Repeated pair comparisons

The original program checks the same pair more than once. For example, comparing event 0 with event 1 and event 1 with event 0 is unnecessary.

Correction:

for i in range(len(events)): for j in range(i + 1, len(events)):

This ensures that each unique pair is checked only once.

Result

After correcting all four defects, the program identifies the closest pair of login events correctly.

The closest pair is (0, 5), and the minimum distance is approximately 1.4142.

The brute force closest-pair algorithm has a time complexity of $O(n^2)$, because it compares every possible pair of events.

PAGE 8: A.6 SUMMARY OF THE NINE IDENTIFIED DEFECTS

Introduction

After examining all four programs used in the SentinelOps SOC Alert-Triage system, nine logical defects have been identified. These defects affect the correctness and reliability of the programs. Some errors are related to incorrect loop conditions, while others are caused by wrong return statements, incorrect comparisons, and unnecessary calculations.

Identifying these defects is an important part of debugging because even a small programming mistake can produce incorrect results. The following section summarises all nine defects, their causes, and the corrections required.

Summary of Identified Defects

1. Incorrect loop condition in insertion sort

The original insertion sort program uses j > 0 instead of j >= 0. Because of this, the program does not check the element at index zero. The condition must be changed to j >= 0 to ensure that all previous elements are checked.

2. Incorrect insertion position in sorting

The original program uses alerts[j] = key instead of alerts[j+1] = key. This may overwrite an existing alert and produce an incorrect result. The key must be inserted at the correct next position.

3. Incorrect return value in blocklist searching

The blocklist searching program returns False when an IP address matches an element in the blocklist. This is incorrect because a matching address should return True.

5. Incorrect loop boundary in pattern matching

The pattern-matching program uses range(n-m) instead of range(n-m+1). Due to this mistake, the final valid starting position is not checked. Adding +1 allows the program to find patterns occurring at the end of the log.

6. Incorrect minimum distance initialisation

The closest-pair program initially assigns the minimum distance as zero. Since the distance between different points is positive, the program cannot find a smaller value. The minimum distance should be initialised using float('inf').

7. Incorrect distance comparison

The program uses the greater-than (>) operator instead of the less-than (<) operator when updating the minimum distance. The condition must be corrected to select the smallest distance.

8. Comparing an event with itself

The original closest-pair program includes comparisons where both indexes are equal. This produces a distance of zero and does not help identify two different events. The program must exclude self-comparisons.

9. Repeated pair comparisons

The original program calculates the distance between the same two events more than once. For example, it checks both (0, 1) and (1, 0). Starting the inner loop from i + 1 avoids repeated comparisons.

Overall Result

A total of nine logical defects have been identified across the four algorithms.

Algorithm	Number of Defects
Insertion Sort	2
Blocklist Searching	2
Pattern Matching	1
Closest Pair	4
Total	**9**

After correcting these defects, all four programs can produce the expected outputs for the given test data. These corrections improve the accuracy and reliability of the SentinelOps SOC Alert-Triage system.

9: B. ROOT CAUSE ANALYSIS AND STEP-BY-STEP EXECUTION

B.1 Introduction

The second part of the assessment focuses on analysing the root causes of the nine defects identified in the previous section. Root cause analysis means finding the actual reason behind an error instead of simply correcting the incorrect output.

In software development, debugging is an important process used to identify, understand, and correct errors in a program. A program may contain syntax errors, runtime errors, or logical errors. In the SentinelOps system, the identified problems are mainly logical errors. The programs can execute, but they may produce incorrect results because of mistakes in conditions, loops, and calculations.

Each algorithm is examined by following its execution step by step. The input values are checked, the important statements are traced, and the actual behaviour is compared with the expected behaviour. This process helps developers understand why an error occurs and how it affects the complete program.

B.2 Root Cause Analysis of Insertion Sort

The first algorithm is insertion sort, which is used to arrange security alerts according to their severity. The main cause of the error is the incorrect loop condition and incorrect insertion position.

The original program uses $j > 0$, which prevents the algorithm from checking the element at index zero. When the selected alert has a higher severity than all previously processed alerts, the program must move the key to the first position. However, the incorrect condition stops the comparison before reaching index zero.

Another error occurs when the program inserts the selected alert into alerts[j] instead of alerts[j+1]. This may overwrite an existing element and result in incorrect alert ordering.

Step-by-step execution:

Consider the input severity values:

[3, 9, 5, 9, 1]

1. Initially, the first alert with severity 3 is considered sorted.
2. The second alert with severity 9 is selected.
3. Since 9 is greater than 3, the alert with severity 3 must move to the right.
4. The alert with severity 9 must be inserted at the first position.
5. The same process continues for the remaining alerts.
6. Finally, the alerts should be arranged in descending order while maintaining the original order of equal severity values.

The corrected program uses j >= 0 and alerts[j+1] = key to solve these problems.

Root Cause: Incorrect loop boundary and incorrect array insertion index.

10: B.3 ROOT CAUSE ANALYSIS OF BLOCKLIST SEARCHING

Introduction

Blocklist searching is an important operation in the SentinelOps SOC Alert-Triage system. It is used to identify whether an IP address is suspicious or safe. The system maintains a list of blocked IP addresses and compares each incoming IP address with the stored values.

The brute force searching technique checks every element in the list one by one. If a matching IP address is found, the program should immediately return True. If no matching address is found after checking the complete list, the program should return False.

Root Cause Analysis

The original blocklist searching program contains two logical errors.

The first error is an incorrect return value. When the program finds a matching IP address, it returns False instead of True. This creates an incorrect result even when the required IP address is present in the blocklist.

The second error is the incorrect placement of the return False statement. It is placed inside the loop, which causes the program to stop searching after checking the first element if it does not match the required IP address.

Because of these mistakes, the program cannot correctly identify all blocked IP addresses.

Step-by-Step Execution

Consider the following blocklist:

blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]

The system receives the IP address:

ip = "10.0.0.9"

The execution process is explained below:

1. The program starts searching from the first element of the blocklist.
2. It compares 10.0.0.5 with 10.0.0.9.
3. Both addresses are different, so the program should continue searching.
4. The next element is 10.0.0.9.
5. The program compares the two addresses and finds a match.
6. It should return True because the IP address is present in the blocklist.

If the requested IP address is 8.8.8.8, the program must examine all three elements. Since no matching address is found, it should return False.

Corrected Program

def is_blocked(blocklist, ip): for address in blocklist: if address == ip: return True return False

Result

After correcting the return statements and their placement, the blocklist searching program produces the correct output. It identifies blocked IP addresses accurately and continues searching until a match is found or the complete list has been checked.

Root Cause: Incorrect return value and incorrect placement of the return statement inside the loop.

11: B.4 ROOT CAUSE ANALYSIS OF PATTERN MATCHING

Introduction

Pattern matching is an important operation in the SentinelOps SOC Alert-Triage system. It is used to identify suspicious sequences of characters in security log files. These sequences are called attack signatures and may indicate possible malicious activities.

The brute force pattern-matching algorithm compares the given pattern with the log file at every possible starting position. It checks the characters one by one. If all characters match, the starting position is stored in the result list.

Root Cause Analysis

The original pattern-matching program contains one major logical defect related to the loop boundary.

The program uses the following statement:

for i in range(n - m):

Here, n represents the length of the log file, and m represents the length of the search pattern.

The problem with this condition is that it does not include the final valid starting position. Therefore, if the attack signature occurs at the end of the log file, the program may fail to identify it.

The correct loop condition is:

for i in range(n - m + 1):

Adding +1 ensures that every valid starting position is checked, including the last possible position.

Step-by-Step Execution

Consider the following input:

log = "FAILFAIXFAIL"pattern = "FAIL"

The length of the log is 12, and the length of the pattern is 4.

The program checks the following positions:

1. At index 0, the characters match the pattern FAIL. Therefore, position 0 is stored.
2. At index 1, the characters do not match completely.
3. At index 2, the characters do not match completely.
4. The program continues checking the remaining starting positions.
5. At index 8, the characters again match the pattern FAIL.
6. Therefore, position 8 is also stored in the result list.

The final result is:

[0, 8]

Corrected Program

def find_signature(log, pattern): n = len(log) m = len(pattern) hits = [] if m == 0: return list(range(n + 1)) for i in range(n - m + 1): j = 0 while j < m and log[i + j] == pattern[j]: j += 1 if j == m: hits.append(i) return hits

Additional Test Cases

The corrected program should also be tested with different inputs.

Test Case 1: Pattern at the beginning

Input: log = "ATTACK123" and pattern = "ATTACK"

Expected Output: [0]

Test Case 2: Pattern at the end

Input: log = "HELLOFAIL" and pattern = "FAIL"

Expected Output: [5]

Test Case 3: Pattern longer than the log

Input: log = "ABC" and pattern = "ABCDE"

Expected Output: []

Result

After correcting the loop boundary, the program successfully identifies all occurrences of the attack signature. It also handles patterns appearing at the beginning or end of the log file.

12: B.5 ROOT CAUSE ANALYSIS OF CLOSEST-PAIR ALGORITHM

Introduction

The closest-pair algorithm is used in the SentinelOps SOC Alert-Triage system to identify two login events that have the minimum distance between them. Each login event is represented using two coordinates, X and Y.

The brute force technique calculates the distance between every possible pair of different events. After calculating all distances, the algorithm selects the pair with the smallest distance.

Root Cause Analysis

The original closest-pair program contains four logical defects. These defects affect the calculation and selection of the minimum distance.

Defect 1: Incorrect minimum distance initialisation

The original program initialises the minimum distance as zero. This is incorrect because the distance between two different points is normally greater than zero.

The correct statement is:

min_dist = float('inf')

This allows the program to update the minimum distance when it finds the first valid pair.

Defect 2: Incorrect comparison operator

The original program uses the greater-than (>) operator instead of the less-than (<) operator.

The correct statement is:

if d < min_dist: min_dist = d

This ensures that the smallest distance is selected.

Defect 3: Comparing an event with itself

The original program compares an event with itself. For example, event 0 is compared with event 0. The distance becomes zero, which is not useful because the objective is to find two different events.

Defect 4: Repeated pair comparisons

The original program checks the same pair more than once. For example, comparing event 0 with event 1 and then event 1 with event 0 is unnecessary.

The corrected loop is:

for i in range(len(events)): for j in range(i + 1, len(events)):

Step-by-Step Execution

Consider the events:

events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]

The program calculates the distance between different pairs of events. It compares each calculated distance with the current minimum distance. Whenever a smaller distance is found, the minimum distance and pair indexes are updated.

Finally, the closest pair is identified as event 0 and event 5.

Expected Result:

Closest Pair: (0, 5)

Minimum Distance: 1.4142

13: B.6 TESTING AND VALIDATION OF CORRECTED PROGRAMS

Introduction

Testing is an important stage in software development. It is used to verify whether a program produces the correct output for different input values. A program should not be considered correct only because it works for one example. It must also be tested with different situations and boundary conditions.

In the SentinelOps system, all four corrected algorithms must be tested using suitable test cases. These test cases help identify errors that may occur when the input data changes.

1. Testing Insertion Sort

The sorting program is tested using different severity values.

Test Case 1: Normal input

Input:

[("A1", 3), ("A2", 9), ("A3", 5)]

Expected Output:

[("A2", 9), ("A3", 5), ("A1", 3)]

Test Case 2: Equal severity values

Input:

[("A1", 5), ("A2", 5), ("A3", 3)]

Expected Output:

[("A1", 5), ("A2", 5), ("A3", 3)]

This test verifies that alerts with equal severity maintain their original order.

2. Testing Blocklist Searching

The blocklist searching program is tested with existing and non-existing IP addresses.

Test Case 1: IP address present

Input:

blocklist = ["10.0.0.5", "10.0.0.9"]ip = "10.0.0.9"

Expected Output:

True

Test Case 2: IP address not present

Input:

blocklist = ["10.0.0.5", "10.0.0.9"]ip = "8.8.8.8"

Expected Output:

False

Test Case 3: Empty blocklist

Input:

blocklist = []ip = "10.0.0.9"

Expected Output:

False

This verifies that the program works correctly even when the blocklist is empty.

3. Testing Pattern Matching

The pattern-matching program is tested using different log contents.

Test Case 1: Pattern at the end

Input:

log = "HELLOFAIL"pattern = "FAIL"

Expected Output:

[5]

Test Case 2: Pattern longer than log

Input:

log = "ABC"pattern = "ABCDE"

Expected Output:

[]

These tests verify that the program checks the final valid position and handles patterns longer than the log correctly.

Conclusion

Testing and validation help confirm that the corrected programs work properly for normal inputs, empty inputs, repeated values, and boundary conditions.

14: B.7 ADDITIONAL TEST CASES AND EXPECTED RESULTS

Introduction

Additional test cases are required to verify the reliability of the SentinelOps system. These tests focus on special situations that may not occur in normal program execution. Such situations are called edge cases or boundary conditions.

Testing these conditions helps ensure that the algorithms do not fail when the input data is small, empty, repeated, or arranged differently.

1. Empty Input Testing

An empty input means that no data is provided to the program.

For blocklist searching, an empty blocklist should return False because no IP address is available for matching.

For pattern matching, an empty log with a non-empty pattern should return an empty list because the pattern cannot occur in the log.

For the closest-pair algorithm, an input containing fewer than two events cannot produce a pair.

2. Pattern at the End of the Log

The pattern-matching algorithm must examine the last valid starting position.

Example:

log = "SECURITYFAIL"pattern = "FAIL"

Expected Output:

[8]

This test verifies that the corrected loop boundary works properly.

3. Pattern Longer Than the Log

A pattern cannot be found if its length is greater than the length of the log.

Example:

log = "ABC"pattern = "SECURITY"

Expected Output:

[]

The program should return an empty list without producing an error.

4. Equal Severity Values

The insertion sort program must maintain the original order of alerts that have equal severity.

Example:

alerts = [("A1", 8), ("A2", 8), ("A3", 4)]

Expected Output:

[("A1", 8), ("A2", 8), ("A3", 4)]

This test verifies the stability of insertion sort.

5. Fewer Than Two Login Events

The closest-pair algorithm requires at least two events to calculate a distance.

Example:

events = [(2, 3)]

Expected Result:

No valid pair exists.

The program should return None for the pair and infinity for the distance, or display an appropriate message.

6. Duplicate Login Coordinates

Two different login events may have the same coordinates. In this situation, the minimum distance is zero.

Example:

events = [(2, 3), (2, 3), (5, 6)]

Expected Output:

Minimum Distance: 0

This verifies that the algorithm can correctly handle duplicate coordinates without comparing an event with itself.

Conclusion

These additional test cases help identify hidden errors and improve the reliability of the system. By testing different input conditions, developers can ensure that the corrected algorithms produce accurate results in practical situations.

PAGE 15: B.8 VALIDATION RESULTS AND CONCLUSION

Introduction

Validation is the process of confirming that the corrected programs satisfy the requirements of the SentinelOps SOC Alert-Triage system. After identifying and correcting the nine defects, all four algorithms must be executed again using the original input data and additional test cases.

The main purpose of validation is to confirm that the corrected programs produce the expected results and handle different input conditions without logical errors.

Validation of Sorting Algorithm

The insertion sort algorithm is executed using the given security alerts. The output is checked to ensure that alerts are arranged in descending order of severity.

The corrected program also maintains the original order of alerts when two or more alerts have equal severity.

Validation Result: PASS

Validation of Blocklist Searching

The blocklist searching algorithm is tested using existing IP addresses, non-existing IP addresses, and an empty blocklist.

The corrected program returns True when the IP address is present and False when it is absent.

Validation Result: PASS

Validation of Pattern Matching

The pattern-matching algorithm is tested using patterns that appear at the beginning, middle, and end of the log file. It is also tested with an empty log and a pattern longer than the log.

The corrected program identifies all valid pattern positions, including the final valid starting position.

Validation Result: PASS

Validation of Closest-Pair Algorithm

The closest-pair algorithm is tested using multiple login events, fewer than two events, and duplicate coordinates.

The corrected program ignores self-comparisons, avoids repeated pair calculations, and identifies the pair with the minimum distance.

Validation Result: PASS

Overall Validation Summary

Algorithm	Main Test	Expected Result	Status
Insertion Sort	Descending severity	Correct order	PASS
Blocklist Search	Existing IP	True	PASS
Pattern Matching	Pattern at end	Position identified	PASS
Closest Pair	Minimum distance	1.4142	PASS

Conclusion

The debugging and validation process successfully identifies and corrects nine logical defects in the SentinelOps SOC Alert-Triage system. The corrected algorithms produce the expected outputs for the given inputs and handle important edge cases.

Testing also confirms that the programs behave correctly under different conditions. Therefore, the corrected implementation provides a reliable foundation for the next stage of the assessment, which focuses on algorithm optimisation and performance improvement.

16: C. ALGORITHM OPTIMISATION AND PERFORMANCE IMPROVEMENT

C.1 Introduction

Algorithm optimisation is the process of improving an algorithm so that it executes faster and uses fewer resources while producing the same correct result. In real-world applications, a program may need to process thousands or even millions of records. If the algorithm is not efficient, the execution time may increase significantly.

The SentinelOps SOC Alert-Triage system processes security alerts, suspicious IP addresses, log files, and login events. As the amount of security data increases, the system must perform these operations efficiently. Therefore, optimisation techniques are required to reduce unnecessary operations and improve overall performance.

In the previous sections, the logical errors in all four algorithms were identified and corrected. However, correcting errors alone does not guarantee that a program is efficient. A program may produce accurate results but still require a large amount of execution time.

This section explains different optimisation techniques that can be applied to the four algorithms used in the SentinelOps system.

Objectives of Optimisation

The main objectives of algorithm optimisation are:

1. To reduce the execution time of programs.
2. To avoid unnecessary comparisons and calculations.
3. To improve performance when processing large input data.

4. To reduce additional memory usage wherever possible.
5. To select suitable algorithms for different types of input.
6. To maintain the correctness of the program after optimisation.

Algorithms Selected for Optimisation

The following four algorithms are considered:

1. Insertion Sort: Optimisation is performed by selecting a suitable sorting method for nearly sorted security alerts.

2. Blocklist Searching: A set data structure is used to improve repeated IP address lookups.

3. Pattern Matching: An efficient pattern-searching technique is considered to reduce unnecessary character comparisons.

4. Closest Pair: An improved closest-pair algorithm is used to reduce the number of distance calculations.

Importance of Optimisation in SentinelOps

Optimisation is important because security monitoring systems must process incoming information quickly. If the system takes too much time to search logs or identify suspicious login activities, security analysts may experience delays in detecting potential threats.

By applying suitable optimisation techniques, the SentinelOps system can process data more efficiently and provide faster results.

PAGE 17: C.2 OPTIMISATION OF INSERTION SORT

Introduction

Insertion sort is a simple sorting algorithm that arranges elements in the required order. In the SentinelOps system, it is used to arrange security alerts according to their severity values.

Insertion sort works by dividing the input into a sorted part and an unsorted part. It selects one element from the unsorted part and inserts it into its correct position in the sorted part.

Working of Insertion Sort

The algorithm starts with the second element and compares it with the previous elements. If the selected element has a higher severity, the previous elements are shifted towards the right. The selected element is then inserted into its correct position.

This process continues until all alerts are arranged in descending order.

Time Complexity

Insertion sort has different time complexities depending on the input arrangement.

- **Best Case:** O(n), when the input is already sorted.
- **Average Case:** $O(n^2)$, when the elements are arranged randomly.
- **Worst Case:** $O(n^2)$, when the elements are arranged in reverse order.
- **Space Complexity:** O(1), because it sorts the elements in place.

Optimisation for Nearly Sorted Data

In security applications, alerts may already be arranged approximately according to their severity. In such situations, insertion sort can perform efficiently because it only needs to move a small number of elements.

For nearly sorted input, insertion sort requires fewer comparisons and shifts than it does for randomly arranged input.

For example, consider the following severity values:

alerts = [9, 8, 7, 6, 5]

This is already arranged in descending order. The insertion sort algorithm only needs to examine each element and does not need to shift any elements.

However, when the input is:

alerts = [3, 9, 5, 8, 1]

The algorithm must perform additional comparisons and shifts to arrange the values correctly.

Using Python's Built-in Sorting Method

For larger datasets, Python's built-in sorting method can be used.

alerts.sort(key=lambda alert: alert[1], reverse=True)

Python uses Timsort, which is designed to work efficiently with real-world data, including partially sorted input. It is stable and has O(n log n) worst-case time complexity.

Advantages of Optimisation

1. Reduces execution time for large datasets.
2. Handles nearly sorted data efficiently.
3. Maintains the original order of equal severity values.
4. Reduces unnecessary comparisons and movements.

Conclusion

Insertion sort is suitable for small and nearly sorted datasets. For larger alert collections, Python's built-in sorting method provides better overall performance.

PAGE 18: C.3 OPTIMISATION OF BLOCKLIST SEARCHING

Introduction

Blocklist searching is used to determine whether an IP address is present in the list of suspicious addresses. The original program uses brute force searching, which checks every element one by one.

Although this method is simple and easy to understand, it becomes slower when the blocklist contains a large number of IP addresses or when many search requests are received.

Therefore, a set data structure can be used to optimise the searching process.

Original Brute Force Method

In the original method, the program compares the requested IP address with every address in the blocklist until a match is found.

For a blocklist containing 1,000 IP addresses, a single search may require checking up to 1,000 elements.

The worst-case time complexity is O(n), where n represents the number of addresses in the blocklist.

Optimised Method Using Set

Python provides a built-in set data structure that supports fast membership checking.

A set stores unique elements and uses hashing to locate values efficiently.

Optimised Code:

blocklist = { "10.0.0.5", "10.0.0.9", "172.16.4.2"}def is_blocked(ip): return ip in blocklist

In this implementation, the blocklist is stored as a set instead of a list.

Whenever an IP address needs to be checked, the program uses the membership operator in.

Working Process

1. The original blocklist is converted into a set.
2. The system receives an IP address.
3. The program checks whether the address exists in the set.
4. If the address is present, the result is True.
5. Otherwise, the result is False.

Time Complexity Comparison

Operation	Brute Force List	Set-Based Search
Single lookup	O(n)	O(1) average
Repeated lookup	O(n) per query	O(1) average per query
Initial construction	O(1)	O(n)
Extra space	O(n) for list	O(n) for set

If the blocklist contains n addresses and the system performs q queries, brute force searching may require O(n × q) time in the worst case.

Using a set requires approximately O(n + q) average time, including the initial construction.

Advantages of Set-Based Searching

1. Provides faster average lookup performance.
2. Reduces repeated comparisons.
3. Suitable for large blocklists.
4. Makes repeated IP address checking more efficient.
5. Provides a simple implementation using Python.

Conclusion

Using a set is an effective optimisation technique for blocklist searching. It improves the performance of repeated IP address lookups and helps the SentinelOps system identify suspicious addresses more quickly.

PAGE 19: C.4 OPTIMISATION OF PATTERN MATCHING

Introduction

Pattern matching is used to search for specific attack signatures in security log files. The original brute force algorithm compares the pattern with the log file character by character at every possible starting position.

This method is easy to implement and understand. However, it may perform many unnecessary character comparisons, especially when the log file and search pattern are large.

To improve performance, an efficient string-searching algorithm such as the Knuth-Morris-Pratt (KMP) algorithm can be used.

Original Brute Force Method

The brute force pattern-matching algorithm checks the pattern at every valid starting position in the log file.

If a mismatch occurs, it moves to the next starting position and starts comparing again from the beginning of the pattern.

The worst-case time complexity is O(n × m), where:

- n = length of the log file.
- m = length of the search pattern.

When the log file is very large, repeated character comparisons can increase execution time.

Optimisation Using KMP Algorithm

The Knuth-Morris-Pratt algorithm improves pattern matching by avoiding unnecessary repeated comparisons.

It uses an additional array called the LPS (Longest Prefix Suffix) array.

The LPS array stores information about the longest proper prefix that is also a suffix for each part of the pattern. When a mismatch occurs, the algorithm uses this information to decide where to continue searching.

As a result, it does not need to restart the comparison from the beginning every time.

Working Process

1. The algorithm receives the security log and attack signature.
2. It first calculates the LPS array for the pattern.
3. It starts comparing the pattern with the log.
4. If characters match, both indexes move forward.
5. If a mismatch occurs, the LPS array is used to avoid unnecessary comparisons.
6. Whenever the complete pattern is found, its starting position is recorded.
7. The algorithm continues searching for other occurrences.

Time Complexity

The KMP algorithm provides the following complexity:

- **Preprocessing Time:** O(m)
- **Searching Time:** O(n)
- **Total Time Complexity:** O(n + m)
- **Space Complexity:** O(m)

Advantages of KMP Optimisation

1. Reduces unnecessary character comparisons.

PAGE 20: C.5 OPTIMISATION OF CLOSEST-PAIR ALGORITHM

Introduction

The closest-pair algorithm is used in the SentinelOps SOC Alert-Triage system to identify two login events that have the minimum distance between them. Each login event is represented using two coordinates, X and Y.

The original brute force algorithm calculates the distance between every possible pair of events. Although this method produces the correct result, it becomes slower when the number of login events increases.

To improve the performance, the divide-and-conquer technique can be used. This technique divides the input into smaller parts, solves the smaller problems, and combines their results to obtain the final answer.

Original Brute Force Method

The brute force method compares every possible pair of different login events. It calculates the distance using the Euclidean distance formula.

Distance = $\sqrt{((x2 - x1)^2 + (y2 - y1)^2)}$

For a small number of events, this method is simple and easy to implement. However, when the input size increases, the number of comparisons also increases rapidly.

The time complexity of the brute force closest-pair algorithm is $O(n^2)$.

Optimisation Using Divide and Conquer

The divide-and-conquer method improves the closest-pair algorithm by dividing the points into two groups.

The algorithm follows these steps:

1. First, the login events are sorted according to their X-coordinates.
2. The events are divided into two approximately equal halves.
3. The closest pair is calculated recursively in the left half.
4. The closest pair is calculated recursively in the right half.
5. The smaller of the two distances is selected.
6. The algorithm then checks a narrow strip around the dividing line to identify a closer pair that may contain one point from each half.
7. Finally, the minimum distance and corresponding pair are returned.

Advantages of Divide and Conquer

1. Reduces the number of distance calculations.

2. Performs better for large datasets.
3. Avoids checking every possible pair.
4. Provides an efficient solution for geometric problems.
5. Helps process large numbers of login events.

Time Complexity

The brute force closest-pair algorithm requires $O(n^2)$ time.

The divide-and-conquer closest-pair algorithm can solve the problem in O(n log n) time when implemented efficiently with suitable sorting and merging procedures.

PAGE 21: C.6 COMPARISON OF ORIGINAL AND OPTIMISED ALGORITHMS

Introduction

After applying optimisation techniques to the four algorithms, it is important to compare their original and optimised implementations. This comparison helps identify how much improvement is achieved in terms of execution time, number of operations, and overall efficiency.

The original algorithms mainly use simple brute force techniques. These methods are easy to understand but may perform unnecessary operations when processing large amounts of data.

The optimised implementations use suitable data structures and improved algorithms to reduce the processing time.

Comparison of Sorting Algorithms

The original insertion sort algorithm has $O(n^2)$ worst-case time complexity. It performs well for small and nearly sorted inputs.

For larger datasets, Python's built-in Timsort is generally more efficient because it has O(n log n) worst-case time complexity and takes advantage of existing order in the input.

Comparison of Blocklist Searching

The original blocklist searching algorithm checks each IP address one by one. Its worst-case time complexity is O(n) for every query.

The optimised implementation uses a set. Set membership checking takes O(1) average time, although the initial construction requires O(n) time.

This makes set-based searching more suitable when many IP address queries are performed.

Comparison of Pattern Matching

The original brute force pattern-matching algorithm has $O(n \times m)$ worst-case time complexity.

The optimised KMP algorithm has O(n + m) time complexity. It uses an LPS array to avoid unnecessary repeated character comparisons.

Therefore, KMP is more suitable for searching large security log files.

Comparison of Closest-Pair Algorithms

The original brute force closest-pair algorithm checks every unique pair of events. Its time complexity is $O(n^2)$.

The optimised divide-and-conquer algorithm reduces the time complexity to O(n log n).

This improvement is especially useful when the number of login events is very large.

Overall Comparison Table

Algorithm	Original Method	Optimised Method
Sorting	Insertion Sort	Python Timsort
Blocklist Search	Linear Search	Set-Based Search
Pattern Matching	Brute Force	KMP
Closest Pair	Brute Force	Divide and Conquer

Conclusion

The comparison shows that optimisation improves the performance of the SentinelOps system. Each optimised algorithm is selected according to the nature of the problem and the amount of input data.

PAGE 22: D. TIME COMPLEXITY AND SPACE COMPLEXITY ANALYSIS

D.1 Introduction

Algorithm analysis is an important part of software development. It helps developers understand how an algorithm behaves when the input size increases.

Two important measurements used in algorithm analysis are time complexity and space complexity.

Time complexity describes how the number of operations increases with the input size. Space complexity describes how much memory an algorithm requires during execution.

In the SentinelOps SOC Alert-Triage system, these measurements are used to compare the original algorithms with their optimised versions.

D.2 Time Complexity

Time complexity represents the amount of computational work performed by an algorithm.

It is commonly expressed using Big-O notation.

The most common time complexities include:

O(1) – Constant Time: The execution work remains approximately constant regardless of input size.

O(log n) – Logarithmic Time: The number of operations increases slowly as the input size increases.

O(n) – Linear Time: The number of operations increases proportionally with the input size.

O(n log n) – Linearithmic Time: The algorithm performs more work than a linear algorithm but is generally efficient for large datasets.

O(n^2) – Quadratic Time: The number of operations grows approximately with the square of the input size.

D.3 Space Complexity

Space complexity measures the additional memory required by an algorithm.

For example, insertion sort modifies the original list and requires O(1) auxiliary space.

Set-based blocklist searching requires additional memory to store the set.

KMP requires an additional LPS array whose size depends on the pattern length.

The divide-and-conquer closest-pair algorithm generally requires O(n) additional memory in an efficient implementation.

D.4 Complexity Comparison

Algorithm	**Original Time**	**Optimised Time**
Insertion Sort	O(n^2)	O(n log n) using Timsort
Blocklist Search	O(n) per query	O(1) average per query
Pattern Matching	O(n × m)	O(n + m)
Closest Pair	O(n^2)	O(n log n)

D.5 Importance of Complexity Analysis

Complexity analysis helps identify which algorithm is suitable for a particular problem.

An algorithm with lower time complexity is generally more suitable for large datasets. However, the actual performance also depends on the programming language, hardware, input arrangement, and implementation details.

Conclusion

Time and space complexity analysis helps compare algorithms and under

PAGE 23: D.6 PERFORMANCE MEASUREMENT OF CLOSEST-PAIR ALGORITHM

Introduction

Performance measurement is used to evaluate how efficiently an algorithm executes for different input sizes. In the SentinelOps system, the closest-pair algorithm is an important operation because it calculates distances between login events.

The brute force algorithm checks every possible unique pair of events. As the number of events increases, the number of distance calculations also increases.

Therefore, performance testing is performed using different input sizes such as 1,000, 4,000, and 8,000 login events.

Number of Distance Calculations

The number of unique pairs in a brute force closest-pair algorithm is calculated using the formula:

Number of Pairs = n(n - 1) / 2

Where n represents the number of login events.

Experiment 1: 1,000 Login Events

Number of pairs:

= 1000 × 999 / 2

= 499,500

Therefore, the brute force algorithm performs 499,500 distance calculations.

Experiment 2: 4,000 Login Events

Number of pairs:

= 4000 × 3999 / 2

= 7,998,000

Therefore, the brute force algorithm performs 7,998,000 distance calculations.

Experiment 3: 8,000 Login Events

Number of pairs:

= 8000 × 7999 / 2

= 31,996,000

Therefore, the brute force algorithm performs 31,996,000 distance calculations.

Performance Comparison Table

Number of Events	Brute Force Distance Calculations
1,000	499,500
4,000	7,998,000
8,000	31,996,000

Analysis

The experimental values show that the number of distance calculations increases rapidly as the input size increases.

When the input increases from 1,000 to 4,000 events, the number of calculations increases significantly.

When the input increases from 4,000 to 8,000 events, the number of calculations increases again.

This demonstrates the quadratic growth of the brute force closest-pair algorithm.

The divide-and-conquer algorithm improves performance by reducing unnecessary pair comparisons. Its time complexity is O(n log n), compared with $O(n^2)$ for the brute force approach.

PAGE 24: D.7 PERFORMANCE MEASUREMENT OF INSERTION SORT

Introduction

Insertion sort is used to arrange security alerts according to their severity. Its performance depends on the arrangement of the input data.

When the input is already sorted or nearly sorted, insertion sort performs fewer comparisons and movements. When the input is randomly arranged or in reverse order, the algorithm performs more operations.

Therefore, performance testing is carried out using nearly sorted and randomly arranged input data.

Experiment 1: Nearly Sorted Input

In this experiment, security alerts are arranged almost in the required descending order.

Example:

alerts = [9, 8, 7, 6, 5, 4, 3]

The algorithm selects one element at a time and compares it with the previous elements.

Since the input is already arranged in descending order, very few movements are required.

For n elements, the number of comparisons is approximately n - 1.

This makes insertion sort efficient for nearly sorted input.

Experiment 2: Randomly Arranged Input

In this experiment, the severity values are arranged randomly.

Example:

alerts = [3, 9, 5, 1, 8, 4, 7]

The algorithm must perform more comparisons and shift elements to their correct positions.

Therefore, the execution time and number of comparisons increase compared with nearly sorted input.

Analysis

The experiment shows that insertion sort is adaptive because its performance depends on the initial arrangement of the input data.

Nearly sorted input requires fewer operations, while reverse-ordered input requires more comparisons and movements.

Python's built-in Timsort is also adaptive and provides O(n log n) worst-case time complexity. It is generally a better choice for larger datasets.

Conclusion

Insertion sort is useful for small and nearly sorted collections of security alerts. However, for large datasets, Python's built-in sorting method is more suitable because it provides better overall performance.

PAGE 25: E. FINAL CORRECTED PYTHON CODE AND TEST HARNESS

E.1 Introduction

The final part of the assessment presents the corrected Python implementation of the four algorithms used in the SentinelOps SOC Alert-Triage system. All nine logical defects identified in the original programs have been corrected.

The final implementation includes insertion sort for arranging security alerts, set-based searching for checking blocked IP addresses, brute force pattern matching for identifying attack signatures, and brute force closest-pair calculation for finding the nearest login events.

The purpose of this implementation is to combine all the corrected algorithms into one Python program and verify their outputs using a test harness.

E.2 Complete Corrected Python Program

```
import math


# 1. Sorting Security Alerts
```

```
def sort_alerts(alerts):
    """
    Sort alerts by severity in descending order.
    Maintains the original order for equal severities.
    """
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1

        while j >= 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1

        alerts[j + 1] = key

    return alerts



# 2. Blocklist Searching
def is_blocked(blocklist, ip):
    """
    Check whether an IP address exists
    in the blocklist using a set.
    """
    blocked_ips = set(blocklist)
    return ip in blocked_ips



# 3. Attack Signature Searching
```

```
def find_signature(log, pattern):
    """
    Find all occurrences of a pattern
    in a security log using brute force.
    """
    n = len(log)
    m = len(pattern)
    hits = []

    if m == 0:
        return list(range(n + 1))

    for i in range(n - m + 1):
        j = 0

        while j < m and log[i + j] == pattern[j]:
            j += 1

        if j == m:
            hits.append(i)

    return hits



# 4. Closest Pair of Login Events
def closest_logins(events):
    """
    Find the closest pair of different login events.
    """
```

```
    if len(events) < 2:
        return None, math.inf

    best_sq = math.inf
    best_pair = None

    for i in range(len(events)):
        for j in range(i + 1, len(events)):

            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]

            distance_sq = dx * dx + dy * dy

            if distance_sq < best_sq:
                best_sq = distance_sq
                best_pair = (i, j)

    return best_pair, math.sqrt(best_sq)



# TEST HARNESS
if __name__ == "__main__":

    # Sorting Test
    alerts = [
        ("A1", 3),
        ("A2", 9),
        ("A3", 5),
```

```
    ("A4", 9),
    ("A5", 1)
]

print("Sorted Alerts:")
print(sort_alerts(alerts))

# Blocklist Test
blocklist = [
    "10.0.0.5",
    "10.0.0.9",
    "172.16.4.2"
]

print("\nBlocklist Results:")
print(is_blocked(blocklist, "10.0.0.9"))
print(is_blocked(blocklist, "10.0.0.5"))
print(is_blocked(blocklist, "8.8.8.8"))

# Pattern Matching Test
log = "FAILFAIXFAIL"
pattern = "FAIL"

print("\nPattern Positions:")
print(find_signature(log, pattern))

# Closest Pair Test
events = [
    (2, 3),
```

```
        (12, 30),
        (40, 50),
        (5, 1),
        (12, 10),
        (3, 4)
    ]

    pair, distance = closest_logins(events)

    print("\nClosest Pair:")
    print(pair)

    print("Minimum Distance:")
    print(round(distance, 4))
```

E.3 Expected Output

```
Sorted Alerts:
[('A2', 9), ('A4', 9), ('A3', 5),
 ('A1', 3), ('A5', 1)]

Blocklist Results:
True
True
False

Pattern Positions:
[0, 8]

Closest Pair:
(0, 5)
```

Minimum Distance:

1.4142

E.4 Explanation

The corrected program successfully performs all four operations. The sorting function arranges security alerts in descending order while maintaining stability. The blocklist function identifies whether an IP address is present in the set. The pattern-matching function searches for all occurrences of an attack signature. Finally, the closest-pair function calculates the minimum distance between different login events.

The test harness executes each function using the given input values and displays the results. This confirms that the corrected programs produce the expected outputs.

Result: All four algorithms execute successfully with the given test data, and the identified nine logical defects are corrected.